

DAY 5 HAC KATHON

Day 5 Testing, Error Handling, and Backend Integration Refinement Documentation

prepared by : laiba gohar

Overview

Today's milestone focused on ensuring the marketplace is fully functional, responsive, and optimized. This included rigorous functional testing, implementing error-handling mechanisms, optimizing performance, and refining backend integration. The goal was to ensure a seamless user experience, fast load times, and reliable API interactions.

Key Objectives

- ❑ Test all components for functionality, performance, and cross-browser/device compatibility.
 - ❑ Implement error-handling mechanisms to provide a smooth fallback UI in case of API or UI failures.
 - ❑ Optimize performance for faster page load times and smooth interactions.
 - ❑ Ensure responsiveness across different screen sizes and browsers.
-

1. Fully Tested and Functional Marketplace Components

Test Coverage

Functional testing was conducted across all major marketplace features, including:

Product listing and display

Category and price range filtering

Product sorting

Cart functionality

Responsive design

Testing Tools Utilized

- **Postman:** Validated API responses, ensuring correct data fetching and handling.
- **Lighthouse:** Used for performance benchmarking to identify areas for improvement (e.g., page load times, accessibility).
- **Cypress:** Used for end-to-end testing to simulate user interactions and validate functional flow.

Test Results

- All test cases were executed successfully and passed without critical issues.
 - No blockers or major bugs were found, ensuring a stable and functional marketplace.
-

2. Clear and User-Friendly Error Handling

Implemented Error Messages

- **API Failures:** If product data fails to load, the system displays a fallback UI with the message “No products available.” Price Range Input Failures:
- If users enter invalid price ranges, the system displays an “Invalid input” message to guide users towards valid values.

Asynchronous Error Handling

- All asynchronous functions were wrapped in try-catch blocks to handle potential errors gracefully.
- Proper logging was added for easier debugging in case of failures.

Fallback UI

- The fallback UI ensures a smooth experience for users even when issues occur, displaying informative error messages and preventing a broken UI.
-

3. Performance Optimization

Optimizations Implemented

- ❑ **Image Compression:** Images were optimized using TinyPNG, and converted to WebP format for faster load times.
- ❑ **Lazy Loading:** Product images are now loaded lazily, reducing initial page load times, especially on slower connections.
- ❑ **Minified and Bundled Files:** CSS and JavaScript files were minified and bundled to reduce the overall size and improve performance. Cache
- ❑ **Strategy:** Implemented caching mechanisms to reduce repeated network requests.

Performance Results

- ❑ With these optimizations, page load times were significantly reduced.
 - ❑ **Lighthouse Performance Scores:** Scores above 90 for performance and accessibility, indicating optimal load times and accessibility.
-

4. Responsive Design

Cross-Browser Testing

- ❑ **Used BrowserStack** to verify the marketplace's responsiveness across **major browsers:** Chrome, Firefox, Safari, and Edge.
- ❑ **Manual Testing:** The design was also tested on multiple physical devices (mobile phones and tablets) to ensure usability.

Mobile Responsiveness

- ❑ A toggleable sidebar was added for mobile users, improving navigation.
- ❑ The layout adapts seamlessly to different screen sizes, ensuring a user-friendly experience across devices.

Results

- ❑ All components of the marketplace are fully responsive, adjusting smoothly to various screen sizes and devices.

Conclusion

By the end of Day 5, the marketplace is now fully optimized, functional, and responsive, with robust error-handling mechanisms and fast load times. The successful completion of testing and optimization ensures that users will have a smooth and enjoyable experience on both desktop and mobile devices.

The next step involves focusing on enhancing the checkout process and preparing for final refinements in the marketplace.